

# Calidad de los datos — Zona Metropolitana de Monterrey

**Este notebook no modifica nada.** Solo mide y reporta. Toda propuesta de limpieza queda documentada al final, sin ejecutarse.

Fuente: `data/processed/atus_zmm.parquet` — los 18 municipios de la ZMM según el Sistema Urbano Nacional, 2019-2024. Se genera con `uv run zona-atus` a partir del consolidado nacional.

Esos 18 municipios son los únicos de Nuevo León que aparecen en ATUS: la cobertura estatal de la encuesta coincide exactamente con la zona metropolitana.

**Ventaja sobre la base nacional: el panel está balanceado.** Los 18 municipios están presentes los seis años, así que las series de tiempo sí son comparables entre sí — a nivel nacional no lo son, porque la cobertura crece de 91 a 198 municipios.

La pregunta que organiza el reporte no es "*¿cuántos nulos hay?*" sino "*¿qué información falta, y falta al azar?*". Son preguntas muy distintas: casi no hay `NaN`, y aun así solo una de cada cuatro filas está completa.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from geostats import rutas, zonas

d = pd.read_parquet(rutas.ATUS_ZMM)
N = len(d)
print(f"{N:,} registros x {len(d.columns)} columnas | {d.ANIO.min()}-{d.ANIO.max()}
      f" | {d.CVE_MUN.nunique()} municipios")
```

379,294 registros x 51 columnas | 2019-2024 | 18 municipios

# Estilo de las gráficas

Identidad GeoStats: Azul Prusia para los datos, Rojo profundo para el énfasis, grafito para el texto y espacio negativo generoso.

Los colores de marca están pensados para impresión, no para marcas sobre fondo claro, así que se validaron antes de usarlos: **Azul Prusia #003153 no pasa** (luminosidad 0.304 contra la banda 0.43–0.77, y croma 0.078 contra el piso 0.10; a ese croma lee como gris). Se conserva su tono exacto —246°— y se sube la luminosidad a 0.45. Los dos rojos pasan sin modificarse, pero entre ellos quedan en  $\Delta E$  14.1 sobre un piso de 15, así que **nunca aparecen como series contiguas**.

```
In [2]: # --- Identidad GeoStats -----
# Azul Prusia es el color de datos de la marca, pero #003153 no sirve como marca
# sobre fondo claro: L=0.304 (la banda válida es 0.43-0.77) y croma 0.078 (piso
# 0.10), o sea que lee como gris. Se conserva su tono (246°) y se sube L a 0.45.
# Los dos rojos pasan sin tocarse, pero entre sí quedan en ΔE 14.1 (piso 15), así
# que nunca se usan como series contiguas.
AZUL      = "#005991" # Azul Prusia ajustado -> datos
ROJO      = "#8B2C1A" # Rojo profundo -> énfasis, títulos, cifras destacadas
OXIDO     = "#B15E2E" # Rojo óxido -> detalle cálido
GRAFITO   = "#2C2C2C" # texto y estructura
GRIS      = "#F2F2F2" # fondos y rejilla
BLANCO    = "#FFFFFF"

# Rampa ordinal del mismo tono 246°, para dimensiones con orden. El croma se
# limita a 0.13 para no salirse del rango de la marca (0.078-0.133): La guía
# pide "sin saturación, sin neón". ΔL de 0.10 entre pasos, sobre el mínimo 0.06.
AZUL_RAMPA = ["#005991", "#1b77b8", "#4195d9"]

# La guía pide Montserrat (títulos), Cormorant Garamond (texto) y Roboto Mono
# (datos). Si no están instaladas, matplotlib baja al siguiente respaldo.
TITULAR    = ["Montserrat", "Helvetica Neue", "Arial", "DejaVu Sans"]
TEXTO      = ["Cormorant Garamond", "EB Garamond", "Georgia", "DejaVu Serif"]
CIFRAS     = ["Roboto Mono", "Menlo", "DejaVu Sans Mono"]

plt.rcParams.update({
    "figure.dpi": 130,
    "figure.facecolor": BLANCO,
    "axes.facecolor": BLANCO,
    "savefig.facecolor": BLANCO,
    "font.family": "sans-serif",
    "font.sans-serif": TITULAR,
    "font.serif": TEXTO,
    "font.monospace": CIFRAS,
    "font.size": 9,
    "axes.edgecolor": GRIS,
    "axes.labelcolor": GRAFITO,
    "axes.labelsize": 9,
    "axes.titlecolor": ROJO, # Los títulos van en rojo profundo
    "axes.titlesize": 11.5,
    "axes.titleweight": "bold",
```

```

"axes.titlelocation": "left",
"axes.titlepad": 16,
"axes.grid": True,
"axes.axisbelow": True,
"grid.color": GRIS,
"grid.linewidth": 1.0,
"xtick.color": GRAFITO,
"ytick.color": GRAFITO,
"xtick.labelsize": 8.5,
"ytick.labelsize": 8.5,
"legend.frameon": False,
"legend.fontsize": 8.5,
"lines.linewidth": 1.8,
})

def limpiar(ax, ejes=("top", "right", "left")):
    """Quita el marco y deja la rejilla como única referencia."""
    for lado in ejes:
        ax.spines[lado].set_visible(False)
    ax.tick_params(length=0)
    for etiqueta in ax.get_xticklabels() + ax.get_yticklabels():
        etiqueta.set_fontfamily("monospace") # Los datos, en monoespaciada
    return ax

def cifra(ax, x, y, texto, color=GRAFITO, **kw):
    """Etiqueta numérica directa, siempre en monoespaciada."""
    return ax.text(x, y, texto, family="monospace", fontsize=8.5,
                  color=color, **kw)

```

## 1. Los NaN mienten

Lo primero que uno hace es `df.isna().sum()`. En esta base, ese conteo es casi vacío: solo tres columnas de texto tienen nulos.

```

In [3]: nulos = d.isna().sum()
nulos = nulos[nulos > 0].to_frame("nulos")
nulos["% del total"] = (nulos.nulos / N * 100).round(2)
nulos

```

```

Out[3]:

```

|                  | nulos  | % del total |
|------------------|--------|-------------|
| <b>CALLE1</b>    | 434    | 0.11        |
| <b>CALLE2</b>    | 5979   | 1.58        |
| <b>CARRETERA</b> | 373649 | 98.51       |

Ese resultado sugiere una base casi perfecta. Es falso.

El INEGI **no usa NaN** : codifica la ausencia de información como un valor numérico válido dentro del propio catálogo. Los códigos están documentados en `docs/diccionario_de_datos.md`.

```
In [4]: # Códigos centinela documentados en el diccionario del INEGI.
CENTINELAS = {
    "HORA": {99: "no especificado"},
    "MINUTOS": {99: "no especificado"},
    "DIA": {32: "no especificado"},
    "DIASEMANA": {8: "no especificado"},
    "SEXO": {1: "se fugó (sexo desconocido)"},
    "ALIENTO": {6: "se ignora"},
    "CINTURON": {9: "se ignora"},
    "EDAD": {0: "se fugó", 99: "no especificado"},
}

filas = [
    {"campo": campo, "código": codigo, "significado": etiqueta,
     "registros": int((d[campo] == codigo).sum())}
    for campo, codigos in CENTINELAS.items()
    for codigo, etiqueta in codigos.items()
]
inventario = pd.DataFrame(filas)
inventario["% del total"] = (inventario.registros / N * 100).round(2)
inventario.sort_values("registros", ascending=False, ignore_index=True)
```

```
Out[4]:
```

|   | campo     | código | significado                | registros | % del total |
|---|-----------|--------|----------------------------|-----------|-------------|
| 0 | CINTURON  | 9      | se ignora                  | 274045    | 72.25       |
| 1 | EDAD      | 99     | no especificado            | 68686     | 18.11       |
| 2 | ALIENTO   | 6      | se ignora                  | 55639     | 14.67       |
| 3 | EDAD      | 0      | se fugó                    | 36973     | 9.75        |
| 4 | SEXO      | 1      | se fugó (sexo desconocido) | 36973     | 9.75        |
| 5 | HORA      | 99     | no especificado            | 0         | 0.00        |
| 6 | DIASEMANA | 8      | no especificado            | 0         | 0.00        |
| 7 | MINUTOS   | 99     | no especificado            | 0         | 0.00        |
| 8 | DIA       | 32     | no especificado            | 0         | 0.00        |

## La comparación

Cuánta información falta según `isna()` contra cuánta falta **de verdad**.

```
In [5]: def marca_centinela(campo):
        marca = pd.Series(False, index=d.index)
```

```

for codigo in CENTINELAS.get(campo, {}):
    marca = marca | (d[campo] == codigo)
return marca

def faltante_real(campo):
    return d[campo].isna() | marca_centinela(campo)

CAMPOS = ["CINTURON", "EDAD", "ALIENTO", "SEXO", "CALLE2"]
comparacion = pd.DataFrame({
    "código centinela": [marca_centinela(c).mean() * 100 for c in CAMPOS],
    "nulo explícito": [d[c].isna().mean() * 100 for c in CAMPOS],
}, index=CAMPOS)
comparacion["faltante real"] = comparacion.sum(axis=1)
comparacion = comparacion.sort_values("faltante real")
comparacion.round(2)

```

Out[5]:

|                 | código centinela | nulo explícito | faltante real |
|-----------------|------------------|----------------|---------------|
| <b>CALLE2</b>   | 0.00             | 1.58           | 1.58          |
| <b>SEXO</b>     | 9.75             | 0.00           | 9.75          |
| <b>ALIENTO</b>  | 14.67            | 0.00           | 14.67         |
| <b>EDAD</b>     | 27.86            | 0.00           | 27.86         |
| <b>CINTURON</b> | 72.25            | 0.00           | 72.25         |

In [6]:

```

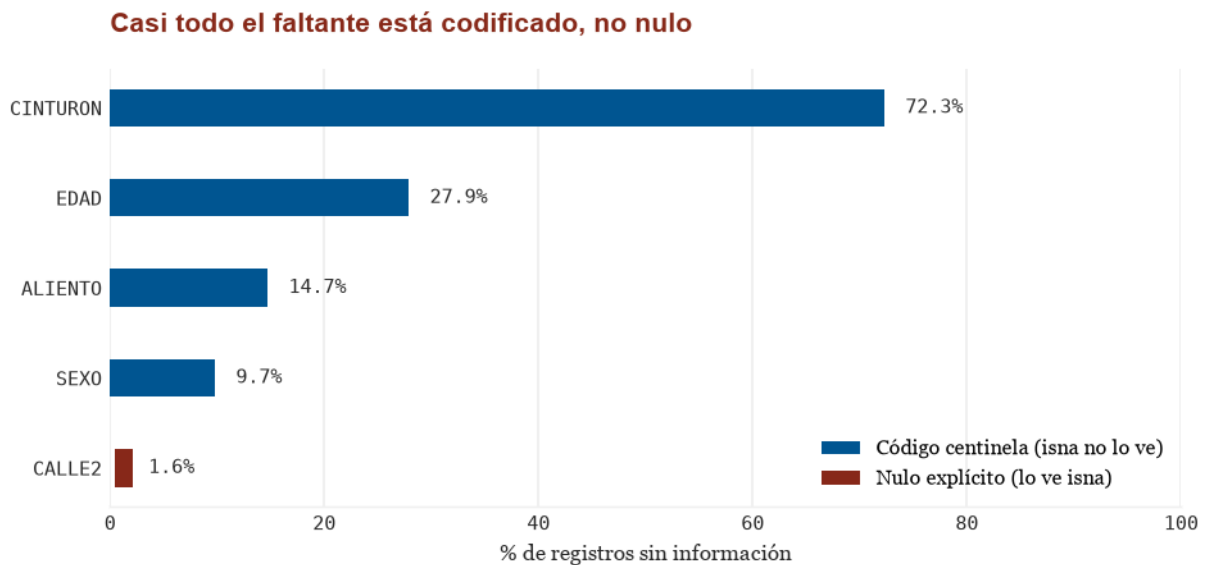
fig, ax = plt.subplots(figsize=(7.4, 3.6))
y = np.arange(len(comparacion))
hueco = 0.5 # el fondo separa los tramos; no se dibuja borde

ax.barh(y, comparacion["código centinela"], height=0.42,
        color=AZUL, label="Código centinela (isna no lo ve)")
ax.barh(y, comparacion["nulo explícito"], height=0.42,
        left=comparacion["código centinela"] + hueco,
        color=ROJO, label="Nulo explícito (lo ve isna)")

for i, v in enumerate(comparacion["faltante real"]):
    cifra(ax, v + 2, i, f"{v:.1f}%", va="center")

ax.set_yticks(y, comparacion.index)
ax.set_xlim(0, 100)
ax.set_xlabel("% de registros sin información", family="serif", fontsize=9.5)
ax.set_title("Casi todo el faltante está codificado, no nulo")
ax.xaxis.grid(True); ax.yaxis.grid(False)
leyenda = ax.legend(loc="lower right")
for t in leyenda.get_texts(): t.set_family("serif"); t.set_fontsize(9)
limpiar(ax)
plt.tight_layout()
plt.show()

```



Cada barra es el faltante total del campo, partido según cómo está expresado. En `CINTURON`, `EDAD`, `ALIENTO` y `SEXO` el tramo rojo no existe: `isna()` devuelve cero y aun así falta hasta el 72.3 % de la información. `CALLE2` es el único caso donde el faltante sí es un nulo de verdad.

`HORA`, `MINUTOS`, `DIA` y `DIASEMANA` también traen centinelas, pero **en la ZMM no aparecen en ningún registro**.

## 2. ¿Cuántos registros están realmente completos?

```
In [7]: CLAVE = ["CINTURON", "ALIENTO", "EDAD", "SEXO", "HORA", "DIA"]
matriz_falta = pd.DataFrame({c: faltante_real(c) for c in CLAVE})
cuantos = matriz_falta.sum(axis=1)

completas = int((cuantos == 0).sum())
print(f"Filas completas en los {len(CLAVE)} campos clave: {completas:,} de {N:,} ({completas/N*100:.1f}%)")

resumen = cuantos.value_counts().sort_index().to_frame("registros")
resumen.index.name = "campos faltantes"
resumen["% del total"] = (resumen.registros / N * 100).round(1)
resumen
```

Filas completas en los 6 campos clave: 96,739 de 379,294 (25.5%)

Out[7]:

|                  | registros | % del total |
|------------------|-----------|-------------|
| campos faltantes |           |             |
| 0                | 96739     | 25.5        |
| 1                | 168809    | 44.5        |
| 2                | 74704     | 19.7        |
| 3                | 2069      | 0.5         |
| 4                | 36973     | 9.7         |

**25.5 % de los registros tienen los seis campos clave** — apenas dos puntos mejor que el 23.5 % nacional. Un análisis que use `dropna()` sobre estos campos se quedaría con una cuarta parte de la base, y no con una cuarta parte representativa: la sección 4 muestra por qué.

### 3. El faltante empeora con el tiempo

Aquí el panel balanceado importa: los 18 municipios están los seis años, así que un cambio en la tasa es un cambio real de captura, no un efecto de cobertura.

In [8]:

```
por_anio = pd.DataFrame(  
    {c: faltante_real(c) for c in ["CINTURON", "EDAD", "ALIENTO", "SEXO"]}  
).assign(ANIO=d.ANIO.values).groupby("ANIO").mean() * 100  
por_anio.round(1)
```

Out[8]:

|      | CINTURON | EDAD | ALIENTO | SEXO |
|------|----------|------|---------|------|
| ANIO |          |      |         |      |
| 2019 | 62.2     | 19.6 | 12.0    | 10.3 |
| 2020 | 74.1     | 24.7 | 13.4    | 10.1 |
| 2021 | 76.8     | 27.1 | 12.9    | 9.8  |
| 2022 | 74.0     | 28.6 | 16.6    | 9.3  |
| 2023 | 71.4     | 29.5 | 13.8    | 9.4  |
| 2024 | 75.4     | 35.9 | 18.5    | 9.7  |

In [9]:

```
# Cuatro campos = cuatro paneles. Una sola serie por panel evita gastar cuatro  
# tonos categóricos, que la paleta de marca no tiene, y deja leer cada curva  
# contra el mismo eje.  
campos = ["CINTURON", "EDAD", "ALIENTO", "SEXO"]  
fig, ejes = plt.subplots(1, 4, figsize=(9.4, 2.9), sharey=True)  
  
for ax, campo in zip(ejes, campos):  
    serie = por_anio[campo]
```

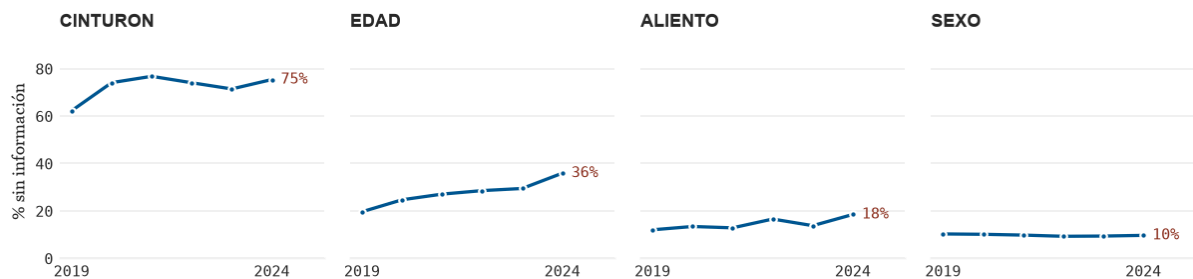
```

ax.plot(serie.index, serie.values, color=AZUL, marker="o", markersize=3.6,
        markerfacecolor=AZUL, markeredgecolor=BLANCO, markeredgewidth=1.2)
cifra(ax, serie.index[-1], serie.iloc[-1], f" {serie.iloc[-1]:.0f}%",
      va="center", ha="left", color=ROJO)
ax.set_title(campo, fontsize=10, pad=10, color=GRAFITO)
ax.set_xticks([2019, 2024])
ax.set_xlim(2018.7, 2025.3)
ax.xaxis.grid(False)
limpiar(ax)

ejes[0].set_ylim(0, 90)
ejes[0].set_ylabel("% sin información", family="serif", fontsize=9.5)
fig.suptitle("La captura se degrada año con año", x=0.005, ha="left",
            color=ROJO, fontsize=11.5, fontweight="bold")
plt.tight_layout(rect=(0, 0, 1, 0.90))
plt.show()

```

La captura se degrada año con año



**EDAD** es el caso más claro: de 19.6 % a 35.9 % de faltante en seis años, casi el doble.

**ALIENTO** sube de 12.0 % a 18.5 %. **CINTURON** se mantiene arriba del 62 % todo el periodo y **SEXO** es el único estable, cerca del 10 %.

Cualquier serie de tiempo sobre uso de cinturón, alcoholemia o edad del conductor en la ZMM mide, en buena parte, **el deterioro del registro**.

## 4. El faltante no es aleatorio: depende de la gravedad

Si los datos faltaran al azar (MCAR), la tasa sería igual en accidentes fatales y en los de solo daños.

```

In [10]: gravedad = d.CLASE.map({1: "Fatal", 2: "No fatal", 3: "Solo daños"})
por_gravedad = (
    pd.DataFrame({c: faltante_real(c) for c in ["ALIENTO", "EDAD", "SEXO", "CINTURON"]
        .assign(gravedad=gravedad.values)
        .groupby("gravedad").mean().loc[["Fatal", "No fatal", "Solo daños"]] * 100
    })
    por_gravedad.round(1)

```

Out[10]:

|                   | ALIENTO | EDAD | SEXO | CINTURON |
|-------------------|---------|------|------|----------|
| <b>gravedad</b>   |         |      |      |          |
| <b>Fatal</b>      | 35.0    | 43.8 | 24.3 | 86.0     |
| <b>No fatal</b>   | 18.4    | 26.5 | 13.4 | 79.3     |
| <b>Solo daños</b> | 14.4    | 27.9 | 9.5  | 71.8     |

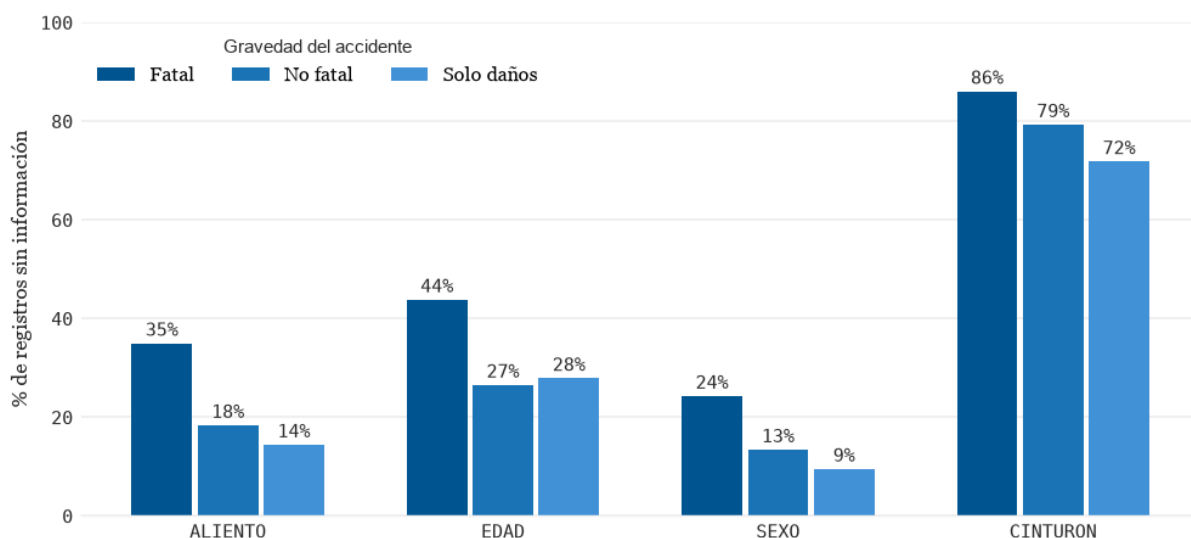
```
In [11]: # La gravedad tiene orden (fatal > no fatal > solo daños), así que se codifica
# con la rampa ordinal de un solo tono, no con tres colores categóricos.
campos = ["ALIENTO", "EDAD", "SEXO", "CINTURON"]
niveles = ["Fatal", "No fatal", "Solo daños"]

fig, ax = plt.subplots(figsize=(7.8, 3.9))
x = np.arange(len(campos))
ancho, hueco = 0.22, 0.02

for i, (nivel, color) in enumerate(zip(niveles, AZUL_RAMPA)):
    valores = [por_gravedad.loc[nivel, c] for c in campos]
    barras = ax.bar(x + (i - 1) * (ancho + hueco), valores, width=ancho,
                    color=color, label=nivel)
    for b, v in zip(barras, valores):
        cifra(ax, b.get_x() + b.get_width() / 2, v + 1.5, f"{v:.0f}%", ha="center")

ax.set_xticks(x, campos)
ax.set_ylim(0, 100)
ax.set_ylabel("% de registros sin información", family="serif", fontsize=9.5)
ax.set_title("Entre más grave el accidente, menos se sabe del conductor")
ax.xaxis.grid(False)
leyenda = ax.legend(ncols=3, loc="upper left", title="Gravedad del accidente")
leyenda.get_title().set_fontsize(8.5)
leyenda.get_title().set_color(GRAFITO)
for t in leyenda.get_texts(): t.set_family("serif"); t.set_fontsize(9)
limpiar(ax)
plt.tight_layout()
plt.show()
```

### Entre más grave el accidente, menos se sabe del conductor



En accidentes **fatales** el aliento alcohólico se ignora en **35.0 %** de los casos, contra 14.4 % en los de solo daños: **2.4 veces más seguido**. El conductor se fugó en 24.3 % contra 9.5 %.

A diferencia de la base nacional, aquí **CINTURON** también muestra el **gradiente** (86.0 % en fatales contra 71.8 % en solo daños), no es plano.

El mecanismo es **MNAR** (*missing not at random*): la probabilidad de que el dato falte depende de aquello que se quiere estudiar. La explicación es plausible — en un accidente fatal el responsable huye más seguido, y cuando huye no hay a quién medirle el aliento ni preguntarle la edad.

**Consecuencia práctica:** eliminar las filas incompletas no es neutral. Borra preferentemente los accidentes fatales con conductor fugado, que son justo el subgrupo de mayor interés. Imputar tampoco es inocuo: los métodos estándar (media, moda, MICE) suponen MAR, y aquí ese supuesto no se cumple.

**EDAD** rompe la monotonía: «Solo daños» (27.9 %) queda por encima de «No fatal» (26.5 %). El patrón limpio es el de **ALIENTO** y **SEXO**. Si se cita el hallazgo, conviene citarlo sobre los extremos.

## 5. El problema mayor: no se captura igual en cada municipio

Esta es la diferencia más grande respecto al reporte nacional. Con un solo estado y 18 municipios se puede ver de dónde viene realmente el faltante.

```
In [12]: d24 = d[d.ANIO == 2024]
tam = d24.groupby("NOM_MUN").size()
grandes = tam[tam > 500].index

por_municipio = (
```

```
pd.DataFrame({c: faltante_real(c)[d.ANIO == 2024] for c in ["CINTURON", "ALIENTO"]
              .assign(NOM_MUN=d24.NOM_MUN.values)
              .groupby("NOM_MUN").mean().loc[grandes] * 100
              }).join(tam.rename("accidentes_2024"))

por_municipio.sort_values("CINTURON", ascending=False).round(1)
```

Out[12]:

|                          | CINTURON | ALIENTO | accidentes_2024 |
|--------------------------|----------|---------|-----------------|
| NOM_MUN                  |          |         |                 |
| Monterrey                | 100.0    | 8.8     | 33568           |
| Santiago                 | 100.0    | 99.8    | 1163            |
| Pesquería                | 99.9     | 9.1     | 1095            |
| Cadereyta Jiménez        | 99.8     | 99.7    | 633             |
| Ciénega de Flores        | 99.7     | 19.8    | 706             |
| Salinas Victoria         | 99.5     | 19.3    | 834             |
| Santa Catarina           | 99.0     | 62.1    | 2730            |
| García                   | 97.8     | 16.2    | 2377            |
| Juárez                   | 92.3     | 37.0    | 1720            |
| Apodaca                  | 85.2     | 23.1    | 3632            |
| Guadalupe                | 80.7     | 25.8    | 2721            |
| General Escobedo         | 42.3     | 42.1    | 3565            |
| General Zuazua           | 40.2     | 30.6    | 697             |
| San Nicolás de los Garza | 13.1     | 13.6    | 8911            |
| San Pedro Garza García   | 10.4     | 10.2    | 6717            |

In [13]: *# Una sola magnitud -> un solo color. El rojo de la marca marca el caso que  
# manda la historia; no es una segunda serie.*

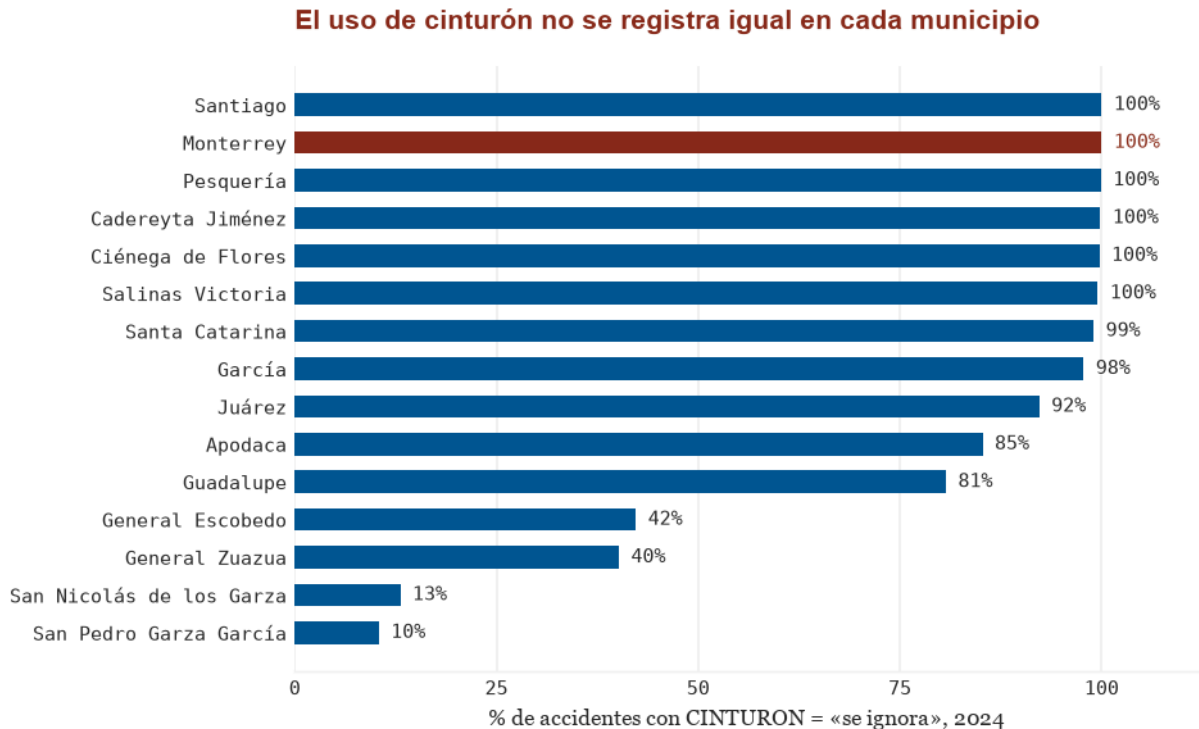
```
orden = por_municipio.sort_values("CINTURON")
colores = [ROJO if m == "Monterrey" else AZUL for m in orden.index]

fig, ax = plt.subplots(figsize=(7.4, 4.6))
y = np.arange(len(orden))
ax.barh(y, orden["CINTURON"], height=0.6, color=colores)

for i, (m, v) in enumerate(zip(orden.index, orden["CINTURON"])):
    cifra(ax, v + 1.5, i, f"{v:.0f}%", va="center",
          color=ROJO if m == "Monterrey" else GRAFITO)

ax.set_yticks(y, orden.index)
ax.set_xlim(0, 112)
ax.set_xticks([0, 25, 50, 75, 100])
ax.set_xlabel("% de accidentes con CINTURON = «se ignora», 2024",
              family="serif", fontsize=9.5)
```

```
ax.set_title("El uso de cinturón no se registra igual en cada municipio")
ax.xaxis.grid(True); ax.yaxis.grid(False)
limpiar(ax)
plt.tight_layout()
plt.show()
```



El rango va de **10.4 % en San Pedro Garza García a 100.0 % en Monterrey**. No es una diferencia de conducta vial: es una diferencia de práctica administrativa. San Pedro y San Nicolás capturan el dato en ~9 de cada 10 accidentes; en el otro extremo hay 7 municipios por encima del 99 %.

**Monterrey está marcado en rojo no por ser el único en ese extremo, sino por su peso: es el 45.4 % de los accidentes de la ZMM.** De sus 172,275 registros, 171,861 dicen «se ignora»: quedan **414 con dato real en seis años**. Los otros 6 municipios por encima del 99 % suman apenas el 5.9% de la base y no mueven el agregado; Monterrey sí. Ese 72 % de faltante que vimos en la sección 1 es, en buena medida, un solo municipio.

```
In [14]: # ¿Es una política estable o cambia de un año a otro?
estabilidad = (
    d.assign(falta=faltante_real("CINTURON"))
    .pivot_table(index="NOM_MUN", columns="ANIO", values="falta", aggfunc="mean")
)
estabilidad["rango"] = estabilidad.max(axis=1) - estabilidad.min(axis=1)
estabilidad.sort_values("rango", ascending=False).round(1)
```

Out[14]:

|         | ANIO                     | 2019  | 2020  | 2021  | 2022  | 2023  | 2024  | rango |
|---------|--------------------------|-------|-------|-------|-------|-------|-------|-------|
| NOM_MUN |                          |       |       |       |       |       |       |       |
|         | Abasolo                  | 0.0   | 100.0 | 83.3  | 36.4  | 66.7  | 33.3  | 100.0 |
|         | García                   | 17.0  | 66.5  | 99.0  | 98.9  | 100.0 | 97.8  | 83.0  |
|         | General Zuazua           | 87.3  | 99.5  | 99.1  | 34.1  | 21.9  | 40.2  | 77.5  |
|         | Guadalupe                | 20.3  | 96.8  | 96.7  | 95.6  | 97.7  | 80.7  | 77.4  |
|         | Cadereyta Jiménez        | 87.7  | 65.0  | 22.5  | 95.9  | 93.8  | 99.8  | 77.3  |
|         | Apodaca                  | 25.7  | 56.2  | 89.7  | 100.0 | 92.8  | 85.2  | 74.3  |
|         | El Carmen                | 32.2  | 63.3  | 98.6  | 100.0 | 73.6  | 98.1  | 67.8  |
|         | Hidalgo                  | 100.0 | 92.9  | 100.0 | 44.4  | 33.3  | 75.3  | 66.7  |
|         | Santa Catarina           | 98.4  | 100.0 | 36.4  | 99.7  | 99.7  | 99.0  | 63.6  |
|         | Juárez                   | 98.5  | 82.6  | 62.4  | 36.2  | 39.3  | 92.3  | 62.4  |
|         | General Escobedo         | 89.6  | 91.1  | 92.9  | 99.9  | 51.8  | 42.3  | 57.6  |
|         | San Nicolás de los Garza | 5.9   | 5.7   | 14.8  | 4.3   | 4.3   | 13.1  | 10.4  |
|         | Santiago                 | 93.2  | 91.0  | 96.5  | 100.0 | 99.9  | 100.0 | 9.0   |
|         | Pesquería                | 91.9  | 96.4  | 99.7  | 99.8  | 99.4  | 99.9  | 8.0   |
|         | San Pedro Garza García   | 6.4   | 7.7   | 7.3   | 4.9   | 11.2  | 10.4  | 6.3   |
|         | Ciénega de Flores        | 99.2  | 99.2  | 100.0 | 99.6  | 96.8  | 99.7  | 3.2   |
|         | Monterrey                | 100.0 | 100.0 | 98.4  | 100.0 | 100.0 | 100.0 | 1.6   |
|         | Salinas Victoria         | 100.0 | 100.0 | 98.8  | 100.0 | 99.7  | 99.5  | 1.2   |

La tabla muestra algo peor que un nivel alto: **inestabilidad**. Guadalupe pasa de 20.3 % en 2019 a 96.8 % en 2020. García va de 17.0 % a 99.0 % en tres años. General Escobedo hace el camino inverso, de 99.9 % en 2022 a 42.3 % en 2024.

Son cambios de régimen administrativo, no de comportamiento. Comparar municipios entre sí, o un municipio consigo mismo entre años, requiere controlar por la tasa de captura.

## 6. Faltantes estructurales: no son errores

Algunos huecos son consecuencia lógica del diseño del formulario y no deben tratarse como datos perdidos.

```
In [15]: print("SEXO=1 ('se fugó') y EDAD=0 ('se ignora porque se fugó') identifican")
print("exactamente los mismos registros:", bool(((d.SEXO == 1) == (d.EDAD == 0)).all))
print(f"registros afectados: {(d.SEXO == 1).sum():,}\n")
```

```
print("CALLE2 (vialidad secundaria) según el tipo de ubicación:")
etiqueta_urbana = d.URBANA.map({0: "0 suburbana", 1: "1 intersección", 2: "2 no int
tabla = pd.crosstab(etiqueta_urbana, d.CALLE2.isna(), normalize="index") * 100
tabla.columns = ["con CALLE2 (%)", "sin CALLE2 (%)"]
tabla.round(1)
```

SEX0=1 ('se fugó') y EDAD=0 ('se ignora porque se fugó') identifican exactamente los mismos registros: True  
registros afectados: 36,973

CALLE2 (vialidad secundaria) según el tipo de ubicación:

Out[15]:

|                   | con CALLE2 (%) | sin CALLE2 (%) |
|-------------------|----------------|----------------|
| URBANA            |                |                |
| 0 suburbana       | 65.8           | 34.2           |
| 1 intersección    | 99.2           | 0.8            |
| 2 no intersección | 89.5           | 10.5           |

URBANA

|                          |      |      |
|--------------------------|------|------|
| <b>0 suburbana</b>       | 65.8 | 34.2 |
| <b>1 intersección</b>    | 99.2 | 0.8  |
| <b>2 no intersección</b> | 89.5 | 10.5 |

CALLE2 ausente en una fila con URBANA = 0 (suburbana) o URBANA = 2 (fuera de intersección) es esperable: no hay segunda vialidad que registrar.

En cambio, **3,072 accidentes marcados como ocurridos en una intersección no tienen segunda calle**. Ahí sí falta información, porque por definición una intersección tiene dos vialidades. Ese es el subconjunto que vale la pena revisar, no el 1.6 % agregado.

## 7. Texto

```
In [16]: texto = []
for c in ["CALLE1", "CALLE2", "CARRETERA"]:
    s = d[c]
    solo_espacios = s.notna() & (s.str.strip() == "")
    texto.append({
        "campo": c,
        "nulos": int(s.isna().sum()),
        "solo espacios": int(solo_espacios.sum()),
        "sin info (%)": round(float((s.isna() | solo_espacios).mean() * 100), 1),
        "valores únicos": int(s.nunique()),
        "con espacios sobrantes": int((s != s.str.strip()).sum()),
    })
pd.DataFrame(texto).set_index("campo")
```

Out[16]:

|  | nulos | solo espacios | sin info (%) | valores únicos | con espacios sobrantes |
|--|-------|---------------|--------------|----------------|------------------------|
|--|-------|---------------|--------------|----------------|------------------------|

| campo     |        |   |      |       |     |
|-----------|--------|---|------|-------|-----|
| CALLE1    | 434    | 0 | 0.1  | 16405 | 22  |
| CALLE2    | 5979   | 0 | 1.6  | 22108 | 135 |
| CARRETERA | 373649 | 0 | 98.5 | 702   | 0   |

La ZMM sale mejor librada que el resto del país en este punto: **no hay un solo valor que sea solo espacios en blanco** (a nivel nacional hay 2,386 en `CARRETERA`), y apenas 157 valores traen espacios sobrantes al inicio o al final.

`CARRETERA` está vacío en el 98.5 % de los registros porque solo aplica a accidentes suburbanos, que aquí son el 1.7 % del total.

## 8. Coordenadas: válidas, pero muy concentradas

```
In [17]: en_zmm = d.LONGITUD.between(-101.0, -99.5) & d.LATITUD.between(25.0, 26.5)
print(f"Fuera del bounding box de la ZMM: {(~en_zmm).sum():,}")
print(f"Exactamente en (0, 0): {(d.LONGITUD == 0) & (d.LATITUD == 0).sum():,}")
print(f"bbox: lon {d.LONGITUD.min():.4f} a {d.LONGITUD.max():.4f} | lat {d.LATITUD.min():.4f} a {d.LATITUD.max():.4f}")

decimales = d.LONGITUD.astype(str).str.split(".").str[1].str.len()
print("\nDecimales en LONGITUD (3 decimales ~ 110 m de error):")
print(decimales.value_counts().sort_index().to_string())
```

```
Fuera del bounding box de la ZMM: 0
Exactamente en (0, 0): 0
bbox: lon -100.8069 a -99.7640 | lat 25.2649 a 26.3562
Decimales en LONGITUD (3 decimales ~ 110 m de error):
LONGITUD
1      1
2     51
3    814
4   10014
5   108878
6    24370
7   234512
8     654
```

```
In [18]: puntos = d.groupby(["LONGITUD", "LATITUD"]).size().sort_values(ascending=False)
print(f"{len(puntos):,} coordenadas únicas para {N:,} accidentes\n")
for umbral in (1, 10, 100):
    cuantos = int((puntos > umbral).sum())
    peso = puntos[puntos > umbral].sum() / N
    print(f" {cuantos:>6,} puntos con más de {umbral:>3} accidentes -> {peso:5.1}")

print("\nPuntos más repetidos:")
for (lon, lat), k in puntos.head(3).items():
    sub = d[(d.LONGITUD == lon) & (d.LATITUD == lat)]
```

```
calle = sub.CALLE1.value_counts().index[0]
print(f" ({lon}, {lat}) n={k:>4} {sub.NOM_MUN.iloc[0]} · {calle}")
```

131,180 coordenadas únicas para 379,294 accidentes

26,259 puntos con más de 1 accidentes -> 72.3% de los registros  
 5,024 puntos con más de 10 accidentes -> 52.3% de los registros  
 365 puntos con más de 100 accidentes -> 17.9% de los registros

Puntos más repetidos:

(-100.2711108, 25.61360694) n= 625 Monterrey · EUGENIO GARZA SADA  
 (-100.3165057, 25.73030358) n= 610 Monterrey · MANUEL L. BARRAGÁN  
 (-100.3093278, 25.7306836) n= 583 San Nicolás de los Garza · FRAY LUIS DE LEÓN

Ninguna coordenada es inválida, pero la concentración es mucho mayor que a nivel nacional: **72.3 % de los accidentes comparte coordenada exacta con otro** (51.3 % nacional), y **365 puntos concentran el 17.9 % de la base** (contra 5.6 % nacional). Solo hay 131,180 coordenadas distintas para 379,294 accidentes.

Tiene sentido: una traza urbana densa, geocodificada al nodo de cada intersección. Los puntos más repetidos corresponden a corredores viales reales —Eugenio Garza Sada, Manuel L. Barragán— no a un vertedero de centroides.

Aun así, para densidad de kernel o detección de *hotspots* esa duplicación exacta infla los picos de forma artificial. Y como las AGEB urbanas del INEGI están delimitadas por ejes de vialidad, estos puntos caen **sobre** las fronteras de AGEB, no dentro de ellas.

Solo 866 registros tienen 3 decimales o menos (~110 m de error o peor).

## 9. Lo que sí está bien

Vale la pena separar los problemas reales de los imaginarios. La coherencia interna de la base es impecable.

```
In [19]: muertos = ["CONDMUERTO", "PASAMUERTO", "PEATMUERTO", "CICLMUERTO", "OTROMUERTO"]
heridos = ["CONDHERIDO", "PASAHERIDO", "PEATHERIDO", "CICLHERIDO", "OTROHERIDO"]
vehiculos = ["AUTOMOVIL", "CAMPASAJ", "MICROBUS", "PASCAMION", "OMNIBUS", "TRANVIA",
             "CAMIONETA", "CAMION", "TRACTOR", "FERROCARRI", "MOTOCICLET",
             "BICICLETA", "OTROVEHIC"]
edad_valida = d.loc[~d.EDAD.isin([0, 99]), "EDAD"]

pruebas = {
    "TOTMUERTOS != suma de componentes": (d[muertos].sum(axis=1) != d.TOTMUERTOS).sum(),
    "TOTHERIDOS != suma de componentes": (d[heridos].sum(axis=1) != d.TOTHERIDOS).sum(),
    "CLASE=Fatal con cero muertos": ((d.CLASE == 1) & (d.TOTMUERTOS == 0)).sum(),
    "CLASE=Solo daños con víctimas": ((d.CLASE == 3) & (d.TOTMUERTOS + d.TOTHERIDOS == 0)).sum(),
    "URBANA y SUBURBANA marcadas a la vez": ((d.URBANA > 0) & (d.SUBURBANA > 0)).sum(),
    "Sin zona asignada": ((d.URBANA == 0) & (d.SUBURBANA == 0)).sum(),
    "Sin ningún vehículo involucrado": (d[vehiculos].sum(axis=1) == 0).sum(),
    "EDAD válida fuera del rango 12-98": (~edad_valida.between(12, 98)).sum(),
    "Llave (ANIO, EDO, MPIO, ID) duplicada": d.duplicated(["ANIO", "EDO", "MPIO", "ID"]).sum(),
    "Municipio fuera de la ZMM": (~d.CVE_MUN.isin(zonas.ZMM_MONTERREY)).sum(),
```

```
}
pd.Series({k: int(v) for k, v in pruebas.items()}, name="violaciones").to_frame()
```

Out[19]:

|                                       | violaciones |
|---------------------------------------|-------------|
| TOTMUERTOS != suma de componentes     | 0           |
| TOTHERIDOS != suma de componentes     | 0           |
| CLASE=Fatal con cero muertos          | 0           |
| CLASE=Solo daños con víctimas         | 0           |
| URBANA y SUBURBANA marcadas a la vez  | 0           |
| Sin zona asignada                     | 0           |
| Sin ningún vehículo involucrado       | 0           |
| EDAD válida fuera del rango 12-98     | 0           |
| Llave (ANIO, EDO, MPIO, ID) duplicada | 0           |
| Municipio fuera de la ZMM             | 0           |

Cero violaciones en las diez pruebas. Los totales cuadran con sus componentes, la clasificación de gravedad es consistente con el conteo de víctimas, la llave no tiene duplicados y ningún registro se coló de fuera de la zona metropolitana.

**El problema de esta base no es la integridad. Es quién llena el formato.**

## 10. Discrepancia con el diccionario

El diccionario del INEGI dice, sobre las coordenadas:

**LONGITUD** — Valor en formato decimal siempre positivo **LATITUD** — Valor en formato decimal siempre negativo

Los datos son lo contrario, y los datos tienen razón: la ZMM está en longitud negativa (oeste) y latitud positiva (norte). La observación del diccionario está invertida. El inventario completo de discrepancias está en `docs/diccionario_de_datos.md`.

```
In [20]: d[["LONGITUD", "LATITUD"]].agg(["min", "max"]).round(4)
```

Out[20]:

|     | LONGITUD  | LATITUD |
|-----|-----------|---------|
| min | -100.8069 | 25.2649 |
| max | -99.7640  | 26.3562 |

# Plan de limpieza propuesto

**Nada de esto está aplicado en este notebook, que solo mide.** Es el inventario de decisiones a tomar, con la recomendación y su razón.

**Ya implementado.** `geostats.limpieza` aplica estas decisiones como funciones explícitas y versionadas, y produce `atus_zmm_limpio.parquet`. Ver `docs/limpieza.md`. Este notebook se queda midiendo la base **sin limpiar**, que es lo que justifica la limpieza.

| # | Hallazgo                                                                                                         | Acción propuesta                                                                                                                        | Por qué                                                                                                                                          |
|---|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Centinelas ( <code>CINTURON=9</code> , <code>ALIENTO=6</code> , <code>EDAD∈{0,99}</code> , <code>SEXO=1</code> ) | Convertir a <code>NA</code> en columnas paralelas ( <code>CINTURON_NA</code> , ...), <b>conservando el código original</b>              | El código distingue " <i>se ignora</i> " de " <i>se fugó</i> ": son mecanismos distintos y colapsarlos a <code>NaN</code> pierde esa información |
| 2 | <code>SEXO=1 ⇔ EDAD=0</code> (36,973 registros)                                                                  | Derivar una bandera explícita <code>CONDUCTOR_FUGADO</code>                                                                             | Es una variable con significado propio, y un predictor de gravedad, hoy escondida dentro de dos catálogos                                        |
| 3 | MNAR por gravedad                                                                                                | <b>No usar <code>dropna()</code> ni imputación estándar</b>                                                                             | El faltante depende de la gravedad; borrar sesga contra los accidentes fatales, e imputar bajo supuesto MAR introduce sesgo silencioso           |
| 4 | Monterrey no registra <code>CINTURON</code> (414 datos reales en 172,275)                                        | <b>Excluir <code>CINTURON</code> de cualquier análisis que incluya a Monterrey</b> , o restringirlo a los municipios que sí lo capturan | Con 45 % de la base sin el dato, un promedio de la ZMM describe a San Pedro y San Nicolás, no a la zona                                          |
| 5 | Rango 10 %–100 % entre municipios, e inestable entre años                                                        | Reportar siempre la tasa de captura junto a cualquier comparación municipal o temporal                                                  | El rango mide práctica administrativa, no conducta vial                                                                                          |
| 6 | <code>EDAD</code> pasa de 19.6 % a 35.9 % de faltante (2019→2024)                                                | Marcar la serie como no comparable entre años sin normalizar                                                                            | El panel está balanceado, así que el deterioro es real y no un efecto de cobertura                                                               |
| 7 | 157 valores de texto con espacios sobrantes                                                                      | <code>str.strip()</code> en una columna derivada                                                                                        | Fragmentan conteos ( <code>"CENTRO"</code> y <code>"CENTRO "</code> cuentan como dos calles)                                                     |
| 8 | Nombres de vialidad sin normalizar                                                                               | Normalizar mayúsculas y acentos <b>solo en una columna derivada</b>                                                                     | Los acentos son parte del nombre real; destruirlos en la original es irreversible                                                                |
| 9 | 72.3 % de coordenadas                                                                                            | Para KDE/ <i>hotspots</i> , agregar por punto o aplicar <i>jitter</i> documentado                                                       | La duplicación exacta infla los picos de densidad                                                                                                |

| #  | Hallazgo                                                                  | Acción propuesta                                            | Por qué                                                                                                                                                    |
|----|---------------------------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | duplicadas; 365 puntos = 17.9 %                                           |                                                             |                                                                                                                                                            |
| 10 | Puntos sobre ejes de vialidad                                             | No asignar AGEB con un <code>within</code> simple           | Las AGEB urbanas se delimitan por ejes de vialidad: los puntos caen en la frontera                                                                         |
| 11 | <del>866 registros con <math>\leq 3</math> decimales</del> — <b>son 3</b> | Marcarlos y excluirlos de análisis a escala de intersección | <b>Corregido:</b> ese conteo mira solo <code>LONGITUD</code> . Con los dos ejes son 3, contra 2.3 esperados por azar — en la ZMM no hay coordenadas burdas |
| 12 | <code>CALLE2</code> ausente en 3,072 intersecciones                       | Marcar como faltante real, no estructural                   | Una intersección tiene dos vialidades por definición                                                                                                       |

## Regla general

Los datos crudos viven en `data/raw/` y no se tocan. Cualquier limpieza produce un artefacto nuevo en `data/processed/`, generado por código versionado y reproducible — igual que hoy hacen `geostats consolidar` y `geostats zonas`.